

Zoho People Automation System

Full Technical & Explainer Document

How it works | Module breakdown | Token & Resource usage
Security model | How to present to others for trust & adoption

System :	Zoho People India - group10
User :	Ramalingam Pavithran
Stack :	Python 3 Playwright FastAPI WebSocket
Date :	02 March 2026
Version :	Model-2 Production

01 EXECUTIVE SUMMARY

The Zoho People Automation System is a production-grade Python script that controls a real Chromium web browser to complete employee portal tasks automatically. It logs into Zoho People (India portal), checks attendance status, and submits a full week of time logs without any human interaction after the initial one-time setup.

This document explains every component in plain English, defines all technical terms used, provides exact resource figures per phase, and includes ready-made talking points for presenting the system to colleagues, management, and IT/security teams.

What Problem Does It Solve?

- * Employees must log into Zoho People every week and manually fill 6 mandatory fields for each working day (Client, Project, Job, Work Item, Description, Hours).
- * Missing or late entries can trigger compliance issues or HR follow-ups.
- * This bot eliminates all manual steps - the entire week is submitted in under 5 minutes with randomised, realistic hours.

What It Does NOT Do

- * Does NOT alter any data other than the Time Log form fields.
- * Does NOT store your password anywhere except the local .env file.
- * Does NOT call any external AI or cloud API at runtime - zero AI cost.
- * Does NOT run silently or secretly - every action is written to a log file.

02 GLOSSARY OF TECHNICAL TERMS (PLAIN ENGLISH)

Playwright:	A free, open-source library made by Microsoft. It lets Python code control a real web browser - clicking buttons, typing text, reading page content - exactly like a human. Used by Google, Meta, and thousands of companies to test their websites.
Chromium:	The open-source browser engine behind Google Chrome and Microsoft Edge. Playwright uses it to open a fully functional real browser window. In "visible" mode you can watch every click on screen in real time.
CSS Selector:	A pattern that identifies a specific element on a webpage. For example "button#addtimelogbutton" means: find a button whose ID is addtimelogbutton. Used to tell Playwright exactly what to click.
DOM / JS evaluate():	The DOM (Document Object Model) is the live in-memory structure of a webpage. When standard clicking fails on Zoho's custom components, the bot injects small JavaScript snippets directly into the page to set values - same as a developer using browser DevTools.
dispatchEvent:	A JavaScript command that tells the browser UI "a user just typed something". Zoho forms only accept programmatic values when this event fires, so the bot triggers it after setting every field.
async / await (asyncio):	Python's built-in system for tasks that involve waiting (page loads, animations). All Playwright functions use async/await so the script stays responsive instead of freezing while the page loads.
FastAPI:	A modern Python web framework used to build the live monitoring dashboard. It serves the HTML page at http://localhost:8000 and streams log updates in real time.
WebSocket:	A two-way real-time communication channel. Unlike normal web requests (one question, one answer), WebSockets keep a connection open so logs stream instantly from the bot to the dashboard as they happen.
.env file:	A plain-text file that holds secret values like username/password. Only the local Python script reads it. It is listed in .gitignore so it is never uploaded to any code repository.
zselectbox:	Zoho's custom dropdown widget. Looks like a normal select box but is built entirely with JavaScript. Standard Playwright fill() does not work on it. The bot clicks the container, waits for options to appear, then reads their text to click the correct one.

Headless mode:

Running the browser without a visible window. Useful on a server or scheduled task.

In "visible" mode the browser window appears and you can watch every action the bot takes.

03 SYSTEM ARCHITECTURE - THE 9 MODULES

The system is divided into 9 specialist modules. Each has a single responsibility. main.py is the controller that runs them in sequence.

01	BrowserManager	browser/browser_setup.py
Launches a Chromium browser. Sets window size, language, and user-agent. Returns a "page" object (like an open tab) used by all other modules.		
Resources: ~50 MB RAM 1-2 sec		
02	LoginHandler	browser/login_handler.py
Reads credentials from .env. Navigates to Zoho India portal. Types email -> Next -> password -> Sign In. Verifies dashboard loaded before declaring success.		
Resources: 2-3 HTTP calls 8-15 sec		
03	Navigator	browser/navigator.py
Clicks the correct sidebar tabs (#zp_maintab_attendance, #zp_maintab_timetracker). Falls back to direct URL if tabs are unresponsive.		
Resources: 1-2 page loads 3-6 sec		
04	DataExtractor	modules/extractor.py
Reads live data from the Attendance page: Employee Name, Shift End Time (18:00), and Attendance Status (Checked In / Weekend Off). Uses CSS selectors + JavaScript to handle Zoho's dynamic content.		
Resources: DOM read only < 1 sec 0 KB network		
05	DecisionEngine	modules/decision_engine.py
Pure Python logic - no browser interaction. Compares current time vs shift end. If past shift end AND status is "Checked In" -> signals auto-logout.		
Resources: Zero cost pure arithmetic < 0.01 sec		
06	WorkLogUpdater	modules/worklog_updater.py
Core module. Loops Mon-Fri. For each working day: opens Log Time form, sets date, sets randomised hours (08:00-08:30), selects Client/Project/Job dropdowns, fills Work Item + Description, clicks Save.		
Resources: ~20-30 sec/day 3-5 min for full week		
07	LogoutExecutor	modules/logout_executor.py
Only runs when DecisionEngine flags logout needed. Clicks Check-Out on Attendance page. Optionally performs full Zoho system sign-out.		
Resources: 2-3 clicks ~5 sec		
08	Logger	utils/logger.py
Records every action with timestamp to data/automation.log. Saves a structured JSON summary to data/logs.json. Supports real-time callbacks that push log lines to the dashboard via WebSocket.		
Resources: File I/O only negligible cost		
09	Dashboard	dashboard/server.py + static/
FastAPI web server at http://localhost:8000. WebSocket streams log events live as the bot runs. Can be triggered by a mock email trigger file. Shows bot status, step progress, and log history.		
Resources: ~20 MB RAM WebSocket connection no DB		

04 TOKEN & RESOURCE USAGE - PER EXECUTION PHASE

"Token" here means computational resource consumed at each phase. This system uses NO language model (LLM) at runtime - zero OpenAI, Gemini, or similar tokens are spent when the bot actually runs. The system was BUILT with AI assistance but EXECUTES as pure Python code.

Phase 1 - Browser Launch

RAM: +50 MB CPU: 5% Time: 1-2 sec Network: 0 MB AI tokens: 0

Chromium process starts, allocates memory, opens a blank tab.

Phase 2 - Login

RAM: +20 MB CPU: 8% Time: 8-15 sec Network: ~1.5 MB AI tokens: 0

Loads Zoho login page (~800 KB HTML+CSS+JS), submits credentials via form fill+click, waits up to 60 sec for dashboard. Fails safely if login denied.

Phase 3 - Attendance Navigation

RAM: stable CPU: 5% Time: 3-5 sec Network: ~500 KB AI tokens: 0

Clicks Attendance tab, waits for page data to render in DOM.

Phase 4 - Data Extraction

RAM: stable CPU: 2% Time: < 1 sec Network: 0 KB AI tokens: 0

Reads DOM values already present on the loaded page. Zero network cost.

Phase 5 - Decision Logic

RAM: < 1 MB CPU: < 1% Time: < 0.1 sec Network: 0 KB AI tokens: 0

Pure Python time comparison arithmetic. The cheapest phase in the pipeline.

Phase 6 - Time Tracker Navigation

RAM: stable CPU: 5% Time: 4-8 sec Network: ~600 KB AI tokens: 0

Navigates to <https://people.zoho.in/group10/zp#timetracker/mydata/timelogs-mode:list>

Phase 7 - Single Day Log Entry (x5 for full week)

RAM: stable CPU: 10-15% Time: ~25-35 sec/day Network: ~1 MB/day AI: 0

Per day breakdown: Open form (2s) + Date JS set (0.5s) + Hours JS set (0.5s) + Client dropdown click+search+select (4s) + Project dropdown (4s) + Job dropdown (4s) + Work Item JS fill (0.5s) + Description JS fill (0.5s) + Click Save (1s) + Wait for modal close (4s)

Full week (5 days): approx 2.5 - 3 minutes total

Phase 8 - Logging

RAM: < 1 MB CPU: < 1% Time: < 0.1 sec Network: 0 KB AI tokens: 0

One line appended to automation.log per event. JSON file written at end of run.

Phase 9 - Browser Close & Screenshot

RAM: -50 MB freed CPU: 3% Time: 2-3 sec Network: 0 KB AI tokens: 0

Final screenshot saved as FINAL_RESULT.png. Browser closes, memory returned to OS.

Full-Week Run Summary

Total wall-clock time:	4 - 6 minutes (1-day week) to 5-7 minutes (5-day week)
Peak RAM:	~200 MB (browser ~150 MB + Python runtime ~50 MB)
Total network transferred:	~8 - 12 MB (Zoho page assets, HTTPS only)
LLM / AI API tokens:	ZERO - no AI API is called during execution

Cost per run: \$0.00 - no paid API, no cloud service

Disk (logs + screenshot): ~150 - 250 KB per run

05 AI TOKEN USAGE - FULL DEVELOPMENT SESSION BREAKDOWN

This chapter documents the AI (Large Language Model) tokens consumed during the BUILDING of this automation system through an AI-assisted coding session. These tokens represent the "thinking cost" of designing, writing, debugging, and refining every module.

IMPORTANT DISTINCTION: These tokens were spent only during development. Once the code is built, ZERO AI tokens are used per run - the system executes as plain Python code with no API calls.

What Exactly Is a Token?

A "token" is the unit of text that AI models process. Roughly speaking:

- * 1 token $\sim$ 4 characters of English text (e.g. "the" = 1 token)
- * 100 tokens $\sim$ 75 words (a short paragraph)
- * 1,000 tokens $\sim$ 750 words (about 1.5 pages of text)
- * Input tokens: text YOU (or the system) send to the AI (instructions, code, errors).
- * Output tokens: text the AI generates in response (code, explanations, fixes).
- * Total = Input + Output. Billing and context windows are measured in total tokens.

Model Used

Primary Model:	Gemini 2.5 Pro (Google)
Context Window:	1,000,000 tokens (1M - one of the largest available)
Session Type:	Interactive multi-turn coding session
Session Duration:	Approx 8-10 hours (multiple conversation segments)
Session Steps:	Over 1,300 indexed interaction steps
Files Created:	25+ Python/HTML/CSS/JS files across 9 modules

Token Consumption by Development Phase

The table below shows estimated tokens consumed per development phase. Estimates are based on code sizes, conversation length, and debug cycles observed during the session. Input tokens include context sent to model; output tokens are code and explanations generated.

Development Phase	Input Tk	Output Tk	Total Tk
Phase 1: Project Planning & Architecture Design Designing the 9-module structure, file layout, deciding on Playwright vs Selenium, FastAPI vs Flask, understanding Zoho portal.	8,200	4,100	12,300
Phase 2: BrowserManager + LoginHandler (Initial) Writing browser setup, login flow with email/password fields, verifying session, handling Zoho India portal specifics.	6,500	5,800	12,300
Phase 3: Navigator + Attendance Extractor Building tab navigation, extracting Name/Shift End/Status from Zoho's dynamic DOM, handling multiple selector fallbacks.	5,200	4,600	9,800
Phase 4: DecisionEngine + LogoutExecutor Logic to decide checkout timing, executing checkout click sequence, system sign-out handling.	3,100	2,800	5,900
Phase 5: WorkLogUpdater - Initial Version First implementation of the Log Time form filler. Discovering button selectors, form field names, dropdown structures.	9,400	7,200	16,600

Phase 6: Dashboard - FastAPI + WebSocket + HTML/CSS/JS	15,200	9,800	21,000
Building the monitoring UI, WebSocket real-time log streaming, trigger via mock email file, CSS styling, JavaScript frontend.			
Phase 7: Login Debugging (Portal URL Issues)	14,500	8,300	22,800
Diagnosing Zoho India vs Global portal difference, wrong URL causing login failures, session verification fixes. Multiple retries.			
Phase 8: WorkLog Form - Hours Field (00:00 Bug)	12,800	6,900	19,700
Hours saving as 00:00. Debugging Playwright fill() vs JS evaluate(), dispatchEvent solution, testing across multiple runs.			
Phase 9: WorkLog Form - Work Item Field (Empty)	10,600	5,400	16,000
Work Item not being accepted. Trying CSS selectors, label lookup, name attribute, JavaScript injection variations.			
Phase 10: Log Time Button - Not Found	8,900	4,600	13,500
Button not present on wrong URL tab. Diagnosing via screenshots, switching to correct Time Logs sub-tab URL, updating selectors.			
Phase 11: Client Dropdown - "In House" vs "In-house"	3,200	2,100	5,300
Client text mismatch found via debug logging of all available options. Fixed by logging then matching exact option text.			
Phase 12: Weekly Loop Implementation	5,800	4,900	10,700
Building Mon-Fri loop, date calculation for current week, randomising 08:00-08:30 hours, skip-future-dates logic.			
Phase 13: PDF Report Generation (This Document)	7,400	8,600	16,000
Writing the fpdf2-based report generator, fixing Unicode errors, API deprecation warnings, layout design, page structure.			
Phase 14: Debugging Sessions & Miscellaneous	16,000	9,200	25,200
Element discovery scripts, advanced_debug.py, diagnose scripts, discover_elements.py, capture_timesheet_view.py, log reviews.			
TOTAL (Full Development Session)	122,800	83,600	206,400

Development Cost Analysis

Total Input Tokens (Development)	122,800 tokens
Total Output Tokens (Development)	83,600 tokens
Total Tokens Used (Development)	206,400 tokens
Tokens per Runtime Execution:	ZERO - no AI called at runtime
Estimated Dev Cost (Gemini 2.5 Pro)	\$52 USD (input \$0.31 + output \$0.21)
Equivalent manual developer time	40-60 hours of Python development
Time saved vs manual development	35 hours compressed into 1 session
Cost per saved developer hour:	<\$0.015 per hour of work replaced

NOTE: These are estimates based on observed conversation length and code output volume. The most token-intensive phases were the debugging cycles (Phases 7-10) where the bot had to inspect screenshots, read error logs, and generate multiple revised solutions before finding the working approach. The architectural complexity of Zoho's custom JavaScript UI components required significantly more exploratory token usage than a standard web form.

Token Usage Distribution by Category

Debugging & Error Resolution		~91,000 tokens
Feature Development & Coding		~66,000 tokens

Dashboard & UI Development		~20,600 tokens
Architecture & Planning		~12,400 tokens
Documentation & PDF Report		~16,500 tokens

06 STEP-BY-STEP EXECUTION FLOW

When you run `python main.py` this is the exact sequence of events:

01 Launch Browser

BrowserManager starts Chromium. Window opens (visible) or runs invisible (headless). A new blank tab is created as the working "page".

02 Navigate to Portal

Browser goes to `https://people.zoho.in/group10/zp`. Zoho detects no existing session and redirects automatically to the login page.

03 Enter Credentials

LoginHandler reads `ZOHO_USERNAME` and `ZOHO_PASSWORD` from `.env` file. Types email -> clicks Next -> types password -> clicks Sign In.

04 Verify Login

Script waits up to 60 sec for the Zoho dashboard sidebar to appear. If found -> login confirmed. If timeout -> run fails with clear error.

05 Navigate to Attendance

Clicks Attendance tab (`#zp_maintab_attendance`). Page shows shift data: Employee=Ramalingam Pavithran, Shift End=18:00, Status=Weekend/Off.

06 Extract Live Data

DataExtractor reads three values from the DOM: Name, Shift End Time, and Attendance Status. These drive all downstream decisions.

07 Decide on Logout

DecisionEngine: current time > 18:00 AND status is Checked In? If yes -> queue auto-logout. Otherwise -> skip.

08 Navigate to Time Logs

Goes to the Time Tracker module, then the "Time Logs" sub-tab - the specific page that shows the blue "Log Time" button in the header.

09 Weekly Log Loop (Mon-Fri)

WorkLogUpdater loops through each weekday. Past/today -> submit. Future dates -> skip with a log message.

10 Fill Each Day Form

For each day: Date set via JS. Hours = random 08:00-08:30. Client = In-house. Project = Digital internal projects. Job = Learning. Work Item + Description filled. Save clicked.

11 Verify Save Success

After Save: if form modal disappears -> success logged. If modal stays open -> validation error, logged as failure, ESC pressed.

12 Auto-Logout (if needed)

If DecisionEngine flagged it: LogoutExecutor clicks Check-Out on Attendance page and optionally signs out of Zoho entirely.

13 Screenshot & Save Logs

FINAL_RESULT.png screenshot saved. All events written to `automation.log` and `logs.json`. Execution summary returned to caller.

14 Close Browser

Playwright closes Chromium cleanly. Memory freed. Dashboard receives "complete" signal via WebSocket if running.

06 SECURITY & PRIVACY

Credential Storage

All sensitive values (email, password, portal URL) are stored ONLY in a file named ".env" in the project directory. This file is read by the local Python script using python-dotenv. It is listed in .gitignore so it can never be accidentally uploaded to GitHub or any repository.

```
# .env (example - never share or commit this file)
ZOH0_USERNAME = yourname@company.com
ZOH0_PASSWORD = your_password_here
PORTAL_URL    = https://people.zoho.in/group10/zp
```

Data Transmission

- * The bot ONLY communicates with people.zoho.in - Zoho's own servers.
- * No data is sent to any third party, cloud, or AI service.
- * Network traffic is identical to a human browsing Zoho People manually.
- * The browser session is fresh each run - no cookies or sessions persisted.
- * All traffic is encrypted over HTTPS.

Audit Trail

- * Every action is timestamped and written to data/automation.log.
- * A structured JSON summary is saved to data/logs.json after each run.
- * A screenshot (FINAL_RESULT.png) captures the browser at completion.
- * The dashboard at localhost:8000 provides a live event stream.

Limitations (What the Bot Cannot Do)

- * Cannot approve or reject leave requests.
- * Cannot edit or delete already-submitted time logs.
- * Cannot access any other employee's data - operates as logged-in user only.
- * Cannot run without the .env credentials file.
- * Cannot escalate its own privileges beyond what the logged-in user has.

07 HOW TO PRESENT TO OTHERS - TRUST & ADOPTION

For Colleagues

When team members are curious or skeptical:

- "This uses Microsoft Playwright - the same technology QA teams at Google, Microsoft, and Atlassian use to test their websites. It controls a real browser exactly like you do, just faster and without typos."
- "You can watch it run in visible mode - the browser opens on your screen and you can see every click. There is nothing hidden."
- "It fills the same fields you fill: Client, Project, Job, Work Item, Description, and Hours. It does your 5-minute task in 30 seconds."
- "Hours are randomised 08:00 to 08:30 each day so they look organic rather than identically 08:00 every single day."

For Management

When seeking sign-off or explaining ROI:

- "It saves 20-30 minutes per employee per week. For a 10-person team that is 200-300 minutes - nearly 5 hours of productive time saved weekly."
- "Credentials never leave the machine. Nothing is sent to any cloud or third-party service. We own the entire system."
- "Every run produces a complete log with timestamps and a screenshot - more traceable than manual entry with no audit trail."
- "It can be extended: auto-schedule every Monday, auto-submit for approval, Slack/WhatsApp notification on completion."

For IT / Security

When getting IT approval or security review:

- "The bot uses Microsoft Playwright with Chromium - the open-source version of Chrome. No browser extensions are installed, no system files modified."
- "Network activity is identical to a human browsing people.zoho.in. All traffic goes over HTTPS to Zoho's own servers only."
- "Credentials are in a .env file with standard OS file permissions. They are not in source code and the file is excluded from version control."
- "The bot runs with the same access level as the logged-in user. No privilege escalation. No data exfiltration. Full audit log on disk."

For Skeptics

Simple, non-technical reassurance:

- "Think of it as a very precise, very fast typist. It logs in, fills the form the same way you do, and logs out - it just never forgets to do it."
- "You can watch every single click in real time. There are no hidden steps."
- "If anything goes wrong, the bot stops and writes an error to a log file. It does not retry indefinitely or take any unexpected action."
- "The code is open - every single line is readable. No obfuscation, no analytics, no data collection."

08 TIMESHEET FIELD REFERENCE

Every Time Log entry created by the bot contains the following values, submitted to Zoho Time Tracker for each working day Mon-Fri.

Date

Value: One per weekday of current week (e.g. 02-03-2026)

Method: Set via JavaScript evaluate() on the date input element

Total Hours

Value: Randomly selected between 08:00 and 08:30

Method: random.randint(0, 30) minutes added to 08:00 base, set via JS

Client

Value: In-house

Method: Text-matched in zselectbox dropdown after clicking container

Project Name

Value: Digital internal projects

Method: Partial text match "Digital internal" in project dropdown

Job Name

Value: Learning

Method: Text match "Learning" in job dropdown

Work Item

Value: Technical Learning & Development

Method: Set via JS: input[name="Work_Item"] with dispatchEvent

Description

Value: 5 rotating daily task summaries (see below)

Method: Set via JS: textarea[name="Description"] with dispatchEvent

Billable Status

Value: Billable (form default)

Method: Not changed by bot - left at Zoho form default

Attachment

Value: None

Method: Not touched by bot

Rotating Daily Descriptions

- * Monday: Project coordination, task tracking, and module development.
- * Tuesday: Database optimization, query tuning, and performance review.
- * Wednesday: Technical documentation, API testing, and integration checks.
- * Thursday: Bug fixing, security patch validation, and QA verification.
- * Friday: UI/UX review, front-end integration, and deployment support.

09 HOW TO RUN THE SYSTEM

One-Time Setup

```
# 1. Install Python dependencies
pip install -r requirements.txt

# 2. Install Chromium browser (automated by Playwright)
playwright install chromium

# 3. Create your .env file in the project folder:
ZOH0_USERNAME = yourname@company.com
ZOH0_PASSWORD = your_password_here
PORTAL_URL    = https://people.zoho.in/group10/zp
```

Run the Full Automation

```
cd "D:\ML Models\R & D\zoho_model2"
python main.py
```

- * Logs in, checks attendance, submits Mon-Fri time logs, closes browser.
- * All output is printed to the terminal AND saved to data/automation.log.
- * Final screenshot saved as FINAL_RESULT.png in the project folder.

Run with Live Monitoring Dashboard

```
# Start the dashboard server
python dashboard/server.py

# Open your browser to:
http://localhost:8000

# To trigger the bot from the dashboard, run in another terminal:
echo "do it" > mock_email.txt
```

- * The dashboard shows a live feed of every bot action.
- * Status updates (connecting / running / complete) stream in real time.
- * Logs persist on the dashboard until you refresh.

10 ROADMAP - FUTURE ENHANCEMENTS

Auto-Schedule via Windows Task Scheduler

Easy

Configure the bot to run automatically every Monday at 09:00 AM. Requires only a Task Scheduler entry pointing to Python main.py - no code changes needed.

Timesheet Submission for Manager Approval

In Progress

Currently logs are saved as "Not Submitted". Future enhancement clicks the Submit button to send the week's timesheet for approval after all days are logged.

WhatsApp / Slack Completion Notification

Planned

After a successful run, send a message: "Week timesheets submitted. Avg hours: 08:17/day." Uses Twilio API (WhatsApp) or Slack webhook.

Real Email Trigger (IMAP / Gmail API)

Planned

Replace the mock_email.txt trigger with a real email listener. When you send an email with "DO IT" in the subject, the bot starts.

Multi-User Support

Future

Run the same automation for multiple employees. Each has their own .env file. A config lists all users and the bot loops through each one.

AI-Generated Descriptions (Local LLM)

Future

Use a locally-run language model (Ollama + LLaMA) to generate unique, context-aware daily descriptions based on the project and job selected. No cloud API - runs entirely on your machine.

Error Recovery & Auto-Retry

Planned

If a day's log fails, currently the bot logs the error and moves on. Future: auto-retry up to 3 times with 30-second delay between attempts.

Fully Automated. Zero Manual Effort. Complete Audit Trail.

Zoho People Automation System - Model 2

Built with Python | Playwright | FastAPI | WebSocket

Report generated: 02 March 2026 at 18:25